SAFER: Semantic Assessment of Feasible Explanations for Recourse

Ashwin A[1], Muhammed Ashrah[2], Sahana Karthik[3] and V. Raghava Mutharaju[4]

[1]Cochin University of Science and Technology, Kochi, India
[2]Jamia Millia Islamia, New Delhi, India
[3]VIT University, Vellore, India
[4]Indian Institute of Technology Palakkad, India

Abstract

Counterfactual explanations are the standard for algorithmic recourse, yet they lack formal semantics and post-hoc validation. Existing generators output tabular counterfactuals using binary immutability constraints, failing to capture cross-feature dependencies such as temporal coherence—for example, increasing employment tenure without aging. We bridge Explainable AI and Semantic Web technologies by introducing a post-hoc validation pipeline that lifts tabular counterfactuals into RDF knowledge graphs using a formal OWL ontology and declarative RML mappings, then validates them against a four-layer SHACL constraint framework. Our contribution includes SHACL-SPARQL cross-instance rules for immutability, directional constraints, and jump limits—enforcing conditional constraints that generation-time checks cannot express. The framework is method-agnostic and extensible; domain experts can add new rules without modifying the generator. We demonstrate on the German Credit dataset with DiCE. The pipeline produces structured machine-readable validation reports, enabling formal cross-generator comparison and regulatory audit trails for trustworthy algorithmic recourse.The implementation is available at https://github.com/kracr/semantic-alg-recourse.

Keywords

Counterfactual Explanations, SHACL, Semantic Web, Explainable AI, Algorithmic Recourse, RDF

1. Introduction

Automated decision making systems are increasingly used to determine access to housing, employment and credit. The European Union's General Data Protection Regulation (GDPR) grants an individual not only the right to an explanation for automated decisions but also the right to contest them [1]. Counterfactual explanations have emerged as an intuitive and actionable form of explanation because it answers: *what would need to change for a different decision to be reached?* [2].

The foundational work of Wachter, Mittelstadt, and Russell [1] established counterfactual explanations as a legally viable approach to algorithmic recourse. Motivated by the GDPR's "right to explanation," they argued that meaningful explanations can be provided without revealing a model's internal workings by generating a counterfactual that could flip the prediction while also remaining close to the original instance.

A rich ecosystem of counterfactual generators now exists. DiCE [3] provides diverse counterfactual sets via genetic, random, and KD-tree methods. GenRE [4] introduces counterfactual reasoning using eBERT embeddings. ElliCE [5] provides provably robust algorithmic recourse via Rashomon set approximations. These methods generate multiple valid counterfactuals efficiently.

However, existing methods output counterfactuals as flat CSVs or JSON objects, with no formal semantics and no post-hoc mechanism to verify that the generated explanation is actually coherent. A counterfactual that flips the model's prediction is treated as valid by definition. This is a dangerously narrow view of validity.

ISWC 2025: International Semantic Web Conference, October 26–30, 2025, Baltimore, MD, USA
ashwin.a.nair7@gmail.com (A. A); muhammed2306509@st.jmi.ac.in (M. Ashrah); sahana@vit.ac.in (S. Karthik); raghava@iitpkd.ac.in (V. R. Mutharaju)
0000-0000-0000-0000 (A. A); 0000-0000-0000-0000 (M. Ashrah); 0000-0000-0000-0000 (S. Karthik); 0000-0000-0000-0000 (V. R. Mutharaju)
© 2025 Copyright for this paper by its authors. Use permitted under Creative Commons License Attribution 4.0 International (CC BY 4.0).

Consider a counterfactual increasing the employment tenure from 2 to 5 years while keeping the age unchanged at 25. This is a technically valid counterfactual as age is immutable and employment tenure is mutable but this explanation violates the fundamental laws of temporal causality. The generator cannot distinguish between *what is mathematically possible* and *what is actually possible in the real world*. According to Miller [6],effective explanations are selected for relevance. A counterfactual that is temporally impossible fails this criteria for relevance as it is not a meaningful alternative for the user.

We address these limitations by introducing a semantic validation pipeline grounded in the Semantic Web stack. Our approach makes three contributions:

1. **An OWL ontology for credit recourse** with explicit cross-instance relationships that enable comparison between original and counterfactual instances.
2. **A declarative RDF conversion process** using RML-style mappings that transforms any tabular counterfactual output into RDF knowledge graphs, enabling method-agnostic integration.
3. **A four-layer SHACL validation framework** comprising data integrity, hard immutability, feasibility constraints (directional rules and jump limits), and risk warnings. Our SHACL-SPARQL cross-instance rules enforce constraints that generation-time checks cannot express such as limiting employment tenure jumps and preventing unrealistic savings increases.

Crucially, our framework is **post-hoc** and **method-agnostic**. We do not modify the generator. We simply validate its output. This separation of generation vs. validation is the foundation of trustworthy recourse.

2. Related Work

2.1. Counterfactual Explanations vs. Algorithmic Recourse

Before reviewing the literature, it is essential to distinguish between counterfactual explanations and algorithmic recourse, as these terms are often used interchangeably but represent fundamentally different concepts.

Counterfactual explanations answer the question: *"What would need to be different for the model to make a different decision?"* [1]. They describe the minimal changes required to flip a model's prediction, focusing on *what* set of feature changes that would alter the outcome.

Algorithmic recourse, by contrast, answers: *"What can an individual do to achieve a desirable outcome?"* [7, 8]. Recourse is prescriptive; it focuses on the actionable steps an individual can take, within realistic constraints, to change their circumstances and obtain a favourable decision. Recourse must consider not only feature mutability but also effort, time horizon, dependencies, and the individual's agency. A counterfactual may be technically valid yet provide no practical recourse if the suggested changes are infeasible.

This paper bridges these perspectives: we generate counterfactuals and then validate them against recourse criteria, moving from technical validity to practical actionability.

2.2. Counterfactual Explanations

Counterfactual explanations describe the minimal changes required to flip a model's prediction, answering: *what would need to change for a different decision?* [1]. This formulation was motivated by both legal and practical considerations. The GDPR grants individuals the right to an explanation for automated decisions, yet requiring full model transparency would conflict with trade secret protections. Counterfactual explanations offer a pragmatic middle ground: they provide meaningful recourse without requiring access to the model's internal parameters.

The foundational formulation [1] treats counterfactual generation as an optimization problem that minimizes distance while ensuring validity. This "closest possible world" approach has become the dominant paradigm. However, as subsequent work has shown, minimality alone is insufficient.

A comprehensive survey [2] identifies desirable properties including validity, actionability, sparsity, plausibility, and causality. Validity ensures the counterfactual flips the prediction. Actionability requires changes are under the individual's control. Sparsity favours explanations that change few features. Plausibility requires counterfactuals remain realistic. Causality requires respect for causal relationships. The survey notes an inherent tension: improving diversity may reduce plausibility, and ensuring causality may require sacrificing sparsity.

One of the most influential methods, DiCE [3], argues that a single counterfactual is often insufficient as users may face practical constraints that prevent them from adopting the recommended changes. Providing multiple diverse counterfactuals gives users choice. DiCE balances three objectives: validity, proximity, and diversity, using Determinantal Point Processes (DPPs) to encourage diversity. DiCE allows users to specify immutable features and permitted ranges, but its constraint mechanism remains binary: a feature is either immutable or mutable.

ElliCE [5] addresses model uncertainty by leveraging Rashomon sets (models with similar predictive performance) to generate counterfactuals that remain valid across multiple models. However, the emphasis on robustness comes at the expense of diversity and user-centered actionability. GenRE [4] uses generative models to produce counterfactuals through forward sampling, offering advantages in plausibility and computational efficiency, though ensuring actionability remains challenging.

Miller [6] provides a foundational perspective on explainable AI, arguing that explanations are fundamentally social processes involving communication between explainer and explainee. This social framing reinforces the distinction between technical validity and practical actionability.

2.3. Algorithmic Recourse

Algorithmic recourse extends the counterfactual paradigm by emphasising the individual's capacity to act. Where counterfactual explanations answer "what would need to change," recourse answers "what can the individual do to achieve a desirable outcome?" This shift introduces additional considerations: feasibility, agency, and the temporal and effort costs of implementing changes.

Upadhyay et al. [7] formalise recourse as a decision-making problem that accounts for individual preferences and constraints. They introduce "recourse-aware" models that not only provide explanations but also guide individuals toward actionable paths. Their work emphasises that recourse must be personalised.

The counterfactual literature emphasises actionability, ensuring that suggested changes are under the individual's control [2]. Yet the field has largely failed to operationalise this concept beyond binary mutability. A feature is actionable if it is mutable. True actionability requires considering time horizon, effort, dependencies, and temporal coherence. Most generators assume that if a feature is mutable, it is equally actionable regardless of the time, effort, or dependencies involved.

2.4. The Gap: Generation is Not Validation

A common thread running through the literature is that generators enforce constraints *during generation* but provide no mechanism for *post-hoc validation*. DiCE [3] allows users to specify immutable features, but once the counterfactual is generated, there is no independent audit layer to verify it satisfies these constraints.

This is not a theoretical concern. The genetic method in DiCE is a stochastic search algorithm not guaranteed to find a feasible solution. In practice, DiCE may return counterfactuals that violate user-specified constraints. The user has no way to know this without manual inspection.

Moreover, the constraints generators can enforce are limited to those expressible in their optimization objective. A generator can enforce that a feature is immutable, but it cannot enforce that "employment tenure can increase only if age increases proportionally." This is not a limitation of any specific method; it is a fundamental constraint of the generation-first paradigm. What is needed is a separation of concerns: generation and validation should be decoupled.

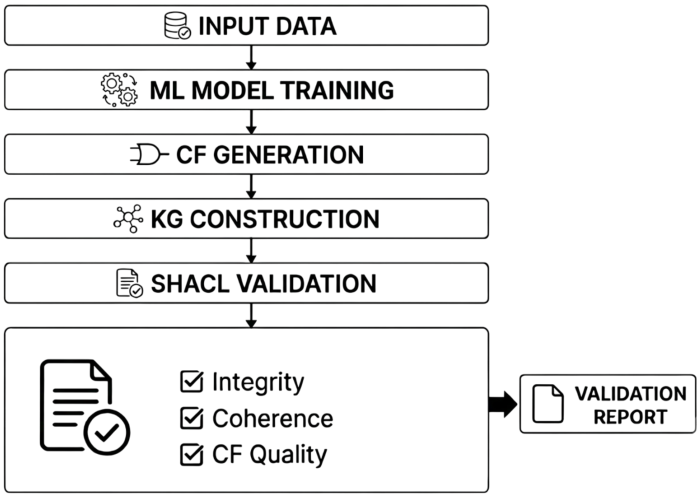


Figure 1: Three-stage pipeline: Knowledge Graph Construction (mapping features to classes and properties using RML), counterfactual generation, and SHACL-based validation with reporting.

2.5. Semantic Web Technologies for Validation

The Semantic Web stack provides a principled foundation for validating structured data. RDF enables interoperable representation. OWL provides formal semantics. SHACL (Shapes Constraint Language) offers a declarative, W3C-standard language for validating RDF graphs against constraint shapes [9, 10].

SHACL has been applied to validate machine learning pipeline outputs [11], ensure data quality [12], and enforce fairness constraints [13]. A key strength is its support for SPARQL-based constraints, which can express complex conditions involving multiple properties and instances, essential for cross-instance validation, such as comparing original and counterfactual values.

To our knowledge, no prior work has applied SHACL to counterfactual explanations, nor has any work introduced cross-instance SPARQL rules for temporal coherence constraints. Our contribution bridges XAI and Semantic Web technologies, bringing the rigour of SHACL validation to the problem of counterfactual recourse, providing a formal, extensible framework that operates post-hoc.

3. Approach

Our approach decouples counterfactual generation from validation. Instead of modifying the generator to enforce additional constraints, we let the generator produce its outputs and independently validate them post-hoc. This separation of concerns allows any counterfactual generator to be integrated, while the validation layer provides a formal, machine-readable assessment of explanation quality.

3.1. Pipeline Overview

The pipeline consists of three broad steps, as illustrated in Figure 1:

1. **Generating counterfactuals:** Candidate counterfactuals are produced using any counterfactual generator, producing explanations that flip the model's prediction.
2. **Mapping features to classes and properties using RML:** Features from the dataset are mapped to an ontology, classifying them according to their mutability, datatypes, and semantic relationships. This step corresponds to the Knowledge Graph Construction phase in the pipeline.
3. **Validating using SHACL and reporting the results:** The generated counterfactuals are validated against a SHACL constraint framework, and the validation results are reported.

3.2. Step 1: Mapping Features to Classes and Properties using RML

Features are classified into immutable, mutable, and ordinal categories. RML (RDF Mapping Language) [14] is used to declaratively map tabular data to RDF triples, specifying classes, properties, datatypes, relationships between original and counterfactual instances, and cross-feature dependencies.

3.3. Step 2: Generating Counterfactuals

Any counterfactual generator can be used to produce candidate explanations. The generator takes as input the original instance and the predictive model, and outputs a set of counterfactual instances that flip the model's prediction. The framework is method-agnostic.

3.4. Step 3: Validating using SHACL and Reporting the Results

The generated RDF graph is validated against a four-layer SHACL constraint framework [9]:

- **Layer 1: Data Integrity** - Validates structural correctness, datatypes, and ranges.
- **Layer 2: Hard Immutability** - Verifies immutable features remain unchanged.
- **Layer 3: Feasibility Constraints** - Enforces directional constraints, jump limits, and dependencies.
- **Layer 4: Counterfactual Quality** - Assesses sparsity, distance, and plausibility.

After validation, results are reported indicating which constraints were satisfied or violated. A human-readable explanation is then generated, listing the changes required and recommending the most feasible path.

4. Evaluation

4.1. Results

We generated 690 counterfactual candidates (≈15 per applicant) for 46 rejected applicants using DiCE with binary immutability for `personal_status`, `foreign_worker`, and `num_dependents`. After applying our SHACL validation framework, **only 262 (37.97%) were fully valid**, while 428 (62.03%) violated at least one constraint.

Table 1 compares valid and invalid counterfactuals across key metrics. Valid counterfactuals are substantially closer to the original instance (distance: 970.95 vs 2353.83) and have lower user-defined cost (21.98 vs 24.11). They also exhibit higher mean age (35.92 vs 30.58) and longer loan duration (19.01 vs 15.83 months), while credit amounts are similar (2640.70 vs 2505.88 DM).

Table 1
Comparison of Valid vs. Invalid Counterfactuals (mean values)

Metric	**Valid** ($n = 262$)	**Invalid** ($n = 428$)
Distance (L1)	970.95	2353.83
Cost	21.98	24.11
Age (years)	35.92	30.58
Credit Amount (DM)	2640.70	2505.88
Duration (months)	19.01	15.83

The violation analysis (Table 2) shows that three rules account for the majority of rejections: *employment cannot decrease* (99 occurrences), *employment increase requires age increase* (76), and *loan amount reduced by >70%* (38). These three rules alone are responsible for over half of all violations. This indicates that DiCE frequently proposes temporally inconsistent changes such as reducing employment

Table 2
Most Frequent Violation Types (single-rule occurrences)

Violation Rule	Frequency
Employment tenure cannot decrease	99
Employment increase requires age increase	76
Loan amount reduced by >70%	38
Checking status jump from <0 to ≥200 DM	23
Duration reduction exceeds 36 months	21
Savings jump from <100 to ≥1000 DM	11
Savings increase requires age increase	10

tenure or increasing it without aging the individual and excessive loan reductions that are unrealistic in practice. These are constraints that binary immutability cannot capture.

At the applicant level, the number of valid counterfactuals per applicant varies considerably. As shown in Figure 2, the distribution is spread widely across applicants, ranging from 0 to 14 valid CFs, with a mode at 7 valid CFs and one outlier applicant reaching 14. All but one applicant had at least one valid CF, meaning that recourse was possible for 45 out of 46 applicants under the semantic constraints.

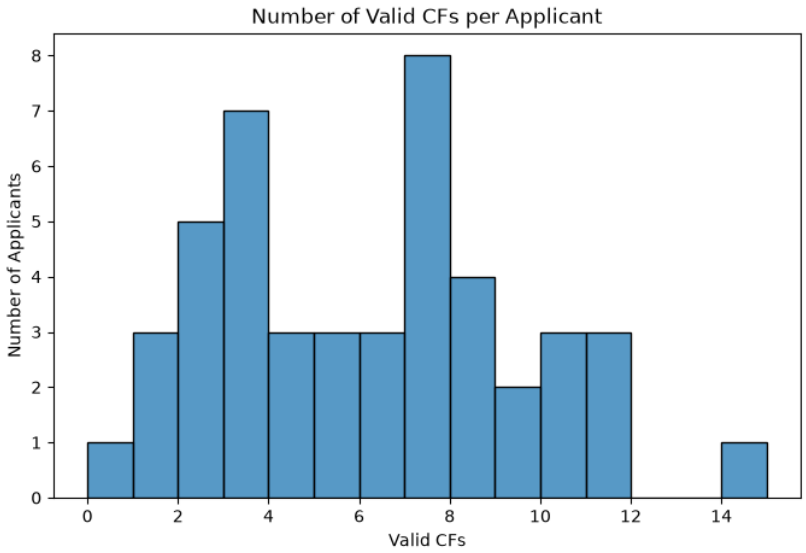


Figure 2: Distribution of valid counterfactuals per applicant. The x-axis shows the number of valid CFs, and the y-axis shows the number of applicants achieving that count.

The immutability rule for `num_dependents` produced zero violations, confirming that the SHACL layer correctly enforces declared immutability when the generator respects it.These results demonstrate that **semantic verification is indispensable** for producing truly actionable counterfactual explanations, as it filters out logically absurd and temporally inconsistent candidates that would otherwise be deemed actionable under a simplistic binary-immutability model.

4.2. Case Studies

To illustrate the practical behaviour of our pipeline, we examine two representative counterfactuals from the generated set.

4.2.1. Case Study 1: A Tight, Realistic Valid Counterfactual

Applicant #136 was originally rejected. Table 3 shows the original profile and a DiCE-generated counterfactual (cf_136_0cdf95) that successfully flips the prediction and passes all SHACL constraints.

Table 3
Valid Counterfactual for Applicant #136

Feature	Original	Counterfactual
Age	22	24
Employment tenure	<1 year	1–4 years
Credit amount (DM)	1409	1800
Duration (months)	10	18
Savings status	<100 DM	<100 DM
Checking status	<0 DM	No checking

This counterfactual changes age from 22 to 24, employment from <1 year to 1-4 years, credit from 1409 to 1800 DM, duration from 10 to 18 months, and checking status from overdrawn to no checking. Critically, the employment increase from <1 year to 1-4 years is accompanied by a 2-year age increase—a perfectly realistic scenario (e.g.: an applicant who was 6 months into a job at age 22 could reasonably have 2.5 years of tenure at age 24). The checking status improves from overdrawn to no checking, which is a decrease of only one ordinal level (1 to 0), well within the allowed 3-level drop. The loan amount and duration both increase, avoiding the reduction caps. No warnings are triggered.

4.2.2. Case Study 2: An Invalid Counterfactual

Applicant #11 was originally rejected due to poor checking status and short employment. Table 4 shows an alternative DiCE-generated counterfactual (cf_11_9fef53) that fails validation.

Table 4
Invalid Counterfactual for Applicant #11

Feature	Original	Counterfactual
Age	19	19
Employment tenure	<1 year	4–7 years
Credit amount (DM)	500	2611
Duration (months)	18	6
Savings status	<100 DM	<100 DM
Checking status	<0 DM	≥200 DM

This counterfactual is rejected for two violations:

1. *Checking status jump from <0 to ≥200 DM is too large.* The checking status improves dramatically (from overdrawn to salary-assignment level) in a single step, which is unrealistic.
2. *Employment increase requires age increase.* Employment tenure jumps from <1 year to 4–7 years while age remains 19, a temporally impossible scenario.

Under a binary-immutability model where only age is locked, this counterfactual would be considered valid. Our semantic validation correctly flags it as infeasible, demonstrating the need for richer constraints.

These case studies highlight two key strengths of our framework: (i) temporal cohesion rules prevent logically absurd scenarios where employment tenure increases without the passage of time, and (ii) jump-limit rules avoid unrealistic step changes in financial status. Together, they illustrate that semantic validation provides a more honest and actionable assessment than generation-time checks alone.

4.3. Discussion

Our results expose a fundamental gap: 62% of DiCE-generated counterfactuals satisfied algorithmic validity (they flip the prediction) but violated semantic feasibility rules. The majority of rejections stem from temporal constraints, specifically employment direction and age cohesion, that generators treat as independent features. This confirms that binary immutability alone is insufficient for actionability.

Valid counterfactuals are significantly closer (970.95 vs 2353.83) and cheaper (21.98 vs 24.11) than invalid ones, demonstrating that semantic rules select for proximal, realistic changes rather than arbitrary solutions. The age-cohesion rule, in particular, eliminates temporally absurd scenarios such as a 19-year-old with 4 to 7 years of employment tenure. This highlights a critical limitation of current generators: they treat features as independent variables, ignoring the causal dependencies that govern real-world recourse.

The immutability rules for personal status, foreign worker, and dependents produced zero violations, confirming correct enforcement. However, the validity rate remains modest at 38%, suggesting that even with proper immutability settings, the genetic search over-adjusts other features and violates feasibility constraints. This exposes a limitation of post-hoc validation: it filters infeasible candidates but does not guide the generator toward feasible regions.

Several limitations warrant mention. Our post-hoc approach filters rather than generates. The cost model is a heuristic lacking empirical calibration. Manual SHACL rule curation requires expert effort. And we model correlation, such as employment increase implies age increase, rather than full causality.

Future work should embed semantic constraints as soft penalties directly into the generator's loss function, integrate causal graphs for richer constraints, and conduct user studies to calibrate the cost model. Applying our framework to other generators would also enable systematic cross-generator comparison.

5. Conclusion

Counterfactual explanations are the standard for algorithmic recourse, yet existing generators optimise for proximity and validity while ignoring the temporal and causal dependencies that determine true actionability.

We introduced a post-hoc validation pipeline that bridges this gap. Using RML-style mappings, we lift tabular counterfactuals into RDF, and validate them against a four-layer SHACL framework. Evaluation on the German Credit dataset with DiCE shows that 62% of generated counterfactuals are semantically infeasible. Valid counterfactuals are significantly closer (970.95 vs 2353.83) and cheaper (21.98 vs 24.11). Critically, temporal cohesion rules eliminate logically absurd scenarios, such as a 19-year-old with 4 to 7 years of employment tenure, while jump limit rules avoid unrealistic step changes in financial status.

The key insight is this: semantic validation is not optional; it is essential for trustworthy recourse. Generators that ignore cross-feature dependencies produce infeasible explanations that mislead users. By leveraging the Semantic Web stack, we provide a formal, extensible layer that ensures counterfactuals are not only mathematically valid but also temporally coherent, realistic, and actionable. Crucially, our framework is method-agnostic, enabling systematic validation across any counterfactual generator.

References

[1] S. Wachter, B. Mittelstadt, C. Russell, Counterfactual explanations without opening the black box: Automated decisions and the GDPR, SSRN Electronic Journal (2017). URL: https://ssrn.com/abstract=3063289. doi:10.2139/ssrn.3063289.

[2] S. Verma, V. Boonsanong, M. Hoang, K. Hines, J. Dickerson, C. Shah, Counterfactual explanations and algorithmic recourses for machine learning: A review, arXiv abs/2010.10596 (2020). URL: https://arxiv.org/abs/2010.10596. doi:10.48550/arXiv.2010.10596.

[3] R. K. Mothilal, A. Sharma, C. Tan, DiCE: Explaining machine learning classifiers through diverse counterfactual explanations, arXiv abs/1905.07697 (2019). URL: https://arxiv.org/abs/1905.07697. doi:10.48550/arXiv.1905.07697.

[4] P. Garg, L. Nagalapatti, S. Sarawagi, From search to sampling: Generative models for robust algorithmic recourse, in: International Conference on Learning Representations (ICLR), 2025. URL: https://proceedings.iclr.cc/paper_files/paper/2025/hash/117308c91189385fd47cfc36d7e6e0e5-Abstract-Conference.html.

[5] B. Turbal, I. Voitsitska, L. Semenova, ElliCE: Efficient and provably robust algorithmic recourse via the rashomon sets, in: Advances in Neural Information Processing Systems (NeurIPS), 2025. The Thirty-ninth Annual Conference on Neural Information Processing Systems.

[6] T. Miller, Explanation in artificial intelligence: Insights from the social sciences, Artificial Intelligence 267 (2019) 1–38. URL: https://www.sciencedirect.com/science/article/pii/S0004370218305988. doi:10.1016/j.artint.2018.07.007.

[7] S. Upadhyay, S. Joshi, H. Lakkaraju, Towards robust and reliable algorithmic recourse, arXiv abs/2102.12646 (2021). URL: https://arxiv.org/abs/2102.12646. doi:10.48550/arXiv.2102.12646.

[8] A.-H. Karimi, G. Barthe, B. Schölkopf, I. Valera, A survey of algorithmic recourse: Definitions, formulations, and solutions, ACM Computing Surveys 55 (2022) 1–36. URL: https://dl.acm.org/doi/10.1145/3527848. doi:10.1145/3527848.

[9] H. Knublauch, D. Kontokostas, Shapes Constraint Language (SHACL), W3C Recommendation, World Wide Web Consortium (W3C), 2017. URL: https://www.w3.org/TR/shacl/.

[10] A. Hogan, E. Blomqvist, M. Cochez, C. d'Amato, G. de Melo, C. Gutierrez, J. E. L. Gayo, S. Kirrane, S. Neumaier, A. Polleres, R. Navigli, T. Pellissier, J. F. Sequeda, S. Staab, A. Zimmermann, Knowledge Graphs, Synthesis Lectures on Data, Semantics, and Knowledge, Morgan & Claypool, 2021. doi:10.2200/S01125ED1V01Y202109DSK022.

[11] C. Debruyne, A. M. B., H. J. Pandit, D. O'Sullivan, Validating machine learning pipelines with SHACL, in: Proceedings of the 4th International Workshop on Semantic Web Enterprise Adoption and Best Practice, 2016.

[12] M. Pavlik, M. K., J. Pokorný, Using SHACL for data quality assessment in rdf stores, Information Systems 86 (2019) 18–29. doi:10.1016/j.is.2019.06.001.

[13] A. Haller, F. Schmidt, M. G., Enforcing fairness constraints with SHACL, in: Proceedings of the 21st International Semantic Web Conference (ISWC), 2022.

[14] A. Dimou, M. V. Sande, P. Colpaert, R. Verborgh, E. Mannens, R. V. de Walle, RML: A generic language for integrated RDF mappings of heterogeneous data, in: Proceedings of the 7th Workshop on Linked Data on the Web (LDOW), 2014.



Dr. V. Raghava Mutharaju
Associate Professor,
Department of Data Science,
Mehta Family School of Data Science and AI,
IIT Palakkad, Kerala. India.

Date: 28/06/2026